

That could be a single letter like 'a', a word like 'hi', or a sentence like 'Hello my friends'.

Writing a string: A Ruby string is defined as a quote sign (") followed by a zero or more letters, numbers or symbols, which are followed by a closing quote sign ("). The shortest possible string is called the empty string: "". It's not uncommon for a single string to contain paragraphs or even pages of text.

Common string methods: Let's experiment with strings and some common methods in IRB.

The `.length` method tells you how many characters (including spaces) are in the string:

```
C:\Users\Administrator>irb
irb(main):001:0> greeting = "hello everyone!"
=> "hello everyone!"
irb(main):002:0> greeting.length
=> 15
```

Often, you'll have a string that you want to break into parts. For instance, imagine you have a sentence stored in a string and want to break it into words:

```
C:\Users\Administrator>irb
irb(main):001:0> sentence = "this is my sample sentence"
=> "this is my sample sentence"
irb(main):002:0> sentence.split
=> ["this", "is", "my", "sample", "sentence"]
```

The `.split` method gives you back an array, which we'll learn about later in this article. It cuts the string wherever it encounters a space (" ") character.

Combining strings and variables: Very often, we want to combine the value of a variable with a string. For instance, let's start with this example string:

"Good morning, Frank!"

When we put that into IRB, it just spits back the same string. If we were writing a proper program, we'd want it to greet users with their name rather than 'Frank'. What we need to do is combine a variable with the string. There are two ways of doing this.

String concatenation: The simplistic approach is called 'string concatenation', which is joining strings together with the plus sign:

```
C:\Users\Administrator>irb
irb(main):001:0> name = "Frank"
irb(main):002:0> puts "Good morning, " + name + "!"
```

In the first line, we set up a variable to hold the name. In the second line, we print the string *"Good morning"*, combined with the value of the variable name and the string *!"*.

String interpolation: The second approach is to use string interpolation, where we stick data into the middle of a string. String interpolation only works on a double-quoted string. Within the string we use the interpolation marker `#{}` . Inside those brackets, we can put any variable or Ruby code, which will be evaluated, converted to a string, and output in that spot of the outer string. Our previous example could be rewritten like this:

```
C:\Users\Administrator>irb
irb(main):001:0> name = "Frank"
irb(main):002:0> puts "Good morning, #{name}!"
```

If you compare the outputs, you'll see that they give the same results. The interpolation style tends to have fewer characters to type, and fewer open/close quotes and plus signs to forget.

4. Numbers

There are two basic kinds of numbers: integers (whole numbers) and floats (have a decimal point). Integers are much easier for both you and the computer to work with. You can use normal math operations with integers including `+`, `-`, `/`, and `*`. Integers have a bunch of methods to help you do math-related things, which you can see by calling `5.methods`.

Repeating instructions: A common pattern in other languages is the 'for' loop, used to repeat an instruction a set number of times. For example, in JavaScript you might write:

```
for(var i = 0; i < 5; i++)
{
  console.log("Hello, World!");
}
```

'For' loops are common, but they're not very readable. Because Ruby's integers are objects, they have methods. One of these is the `.times` method to repeat an instruction a set number of times.

You can rewrite the above loop in Ruby, as follows:

```
5.times do
  puts "Hello, World!"
end
```

In this example, we're using both the `.times` method and what's called a *block*.

5. Blocks

Blocks are a powerful concept used frequently in Ruby. Think of them as a way of bundling up a set of instructions for use elsewhere.